Data Structures for Placements — Day 3: Linked Lists

DSA Prep Notes

oday

Day 3: Linked Lists — Singly, Doubly, Circular

1. Linked Lists — Theory

A **linked list** is a linear data structure where elements (**nodes**) are stored non-contiguously in memory, each node holding data plus a pointer/reference to the next node.

Node structure (singly):

```
struct Node {
    int data;
    Node* next;
};
```

Linked List vs Array — the Fundamental Tradeoff

Aspect	Array	Linked List
Memory	Contiguous	Scattered (nodes linked via pointers)
Access by index	O(1)	O(n) — must traverse from head
Insertion at head	O(n) (shift needed)	O(1)
Insertion at tail	O(1) amortized (vector)	O(1) if tail pointer kept, else O(n)
Insertion at middle	O(n) (shift)	O(n) (traverse) but no shifting once there
Deletion	O(n) (shift)	O(1) once you have the node reference
Extra memory per element	None	Pointer overhead (8 bytes per pointer, x64)
Cache locality	Good (contiguous)	Poor (pointer chasing, scattered memory)

Key insight: linked lists trade O(1) index access for O(1) insertion/deletion *at known positions*. If you need random access, use an array/vector. If you need frequent insertion/deletion (especially at the front, or in the middle once you're already positioned there), a linked list wins.

Three Variants

- **Singly Linked List (SLL):** each node points only to the next node. One-directional traversal.
- **Doubly Linked List (DLL):** each node has both `next` and `prev` pointers. Enables backward traversal and O(1) deletion of a *known* node without needing its predecessor (unlike SLL, where you must walk from head to find the predecessor).
- **Circular Linked List (CLL):** the last node points back to the head instead of null. Used in round-robin scheduling and other wrap-around structures.

Full tested implementations of SLL (insert/delete/search/traverse/reverse iterative+recursive), DLL (insert/delete with O(1) deletion given a node pointer), and CLL (insert/traverse) are in `01_linked_list_basics.cpp`.

Solved Problems — Deep Dive

Problem 1: Detect Cycle in a Linked List (Floyd's Cycle Detection)

Brute force: store visited nodes in a hash set, check if the current node was already seen — O(n) time, O(n) space.

Optimal — Floyd's Tortoise & Hare, O(n) time, O(1) space: move a slow pointer one step and a fast pointer two steps at a time. If there's a cycle, the fast pointer eventually "laps" the slow pointer and they meet inside the loop. If there's no cycle, the fast pointer reaches null first.

Edge cases: empty list (correctly returns false), single node with a self-loop (works correctly when traced through), no cycle at all (fast pointer hits null cleanly, loop exits).

Problem 2: Find the Middle of a Linked List

Brute force: count total nodes in one pass, then traverse `n/2` more steps — two passes total.

Optimal — Slow/Fast Pointer, single pass, O(n) time, O(1) space: advance slow by one step and fast by two steps each iteration; when fast reaches the end, slow is at the middle.

Dry run on [1,2,3,4,5]: slow and fast both start at 1 → slow=2,fast=3 → slow=3,fast=5 → fast->next is null, loop stops. Middle = **3**.

Edge case — even-length lists like [1,2,3,4]: this implementation returns the *second* middle element (3, in a 0-indexed sense the value at index 2) by convention. Worth stating this convention explicitly in an interview — the loop condition can be adjusted (`fast->next && fast->next->next`) if the *first* middle element is required instead.

Problem 3: Merge Two Sorted Linked Lists

Optimal — Iterative with Dummy Node, O(n+m) time, O(1) extra space (reuses existing nodes rather than allocating new ones): maintain a dummy head node and a tail pointer; repeatedly attach whichever list's current node has the smaller value, then advance that list's pointer.

Why the dummy node matters: without it, special-case logic would be needed to decide which list's head becomes the merged list's head. The dummy node sidesteps this — a technique that eliminates edge-case branching and is generally well-regarded in interviews.

Edge cases: one list empty (loop skips entirely; remaining list is attached directly), both lists empty (returns null correctly), duplicate values across lists (using `<=` in the comparison keeps the merge stable — ties favor the first list's node).

Problem 4: Remove Nth Node From End of List

Brute force: find the list length in one pass, compute the target position as `length - n` from the head, then traverse again to that position — two passes.

Optimal — Two Pointer with Gap, single pass, O(n) time, O(1) space: advance a fast pointer `n+1` steps ahead first (creating a gap of size n), then move both fast and slow together until fast reaches null — slow now sits exactly before the node to remove.

Dry run on [1,2,3,4,5], n=2 (remove the 2nd node from the end, i.e. the node holding 4): fast moves 3 steps from a dummy node ahead of the head, landing on node 3. Then both pointers move

together: fast→4, slow→1; fast→5, slow→2; fast→null, slow→3. Node 3 (via `slow`) is immediately before node 4 → node 4 is removed. **Result: [1,2,3,5]**.

Edge cases: removing the head itself (n equals the list length) — this is exactly why the dummy node is essential here too, since slow can legitimately end up positioned "before head" and `slow->next` correctly becomes the new head; n larger than the list length is invalid input and should be validated before running this logic in production code.

Summary Table — Day 3

Problem	Optimal Approach	Time	Space
Detect Cycle	Floyd's Slow/Fast Pointer	O(n)	O(1)
Find Middle Node	Slow/Fast Pointer	O(n)	O(1)
Merge Two Sorted Lists	Dummy Node + Iterative Merge	O(n+m)	O(1)
Remove Nth From End	Two Pointer with Gap	O(n)	O(1)

Structural comparison recap:

Structure	Access	Insert at Head	Insert at Tail	Delete (given node)
Singly Linked List	O(n)	O(1)	O(n)*	O(n)**
Doubly Linked List	O(n)	O(1)	O(n)*	O(1)
Circular Linked List	O(n)	O(n)***	O(n)***	O(n)**

* O(1) if a tail pointer is maintained. ** O(n) because SLL/CLL deletion needs the predecessor node, found by traversal. *** O(n) here since our implementation must locate the last node to update its `next` pointer to the new head.

Code files for today: `01_linked_list_basics.cpp`, `02_linked_list_problems.cpp` — both compiled and tested.

Next session (Day 4): Stacks.